Причинно-следственная карта проекта RUS SLAM

> **Назначение:** свести в одну проверяемую модель требования, механику, электронику, прошивку, ROS 2, навигацию, веб-интерфейс, развёртывание и тесты. **Область:** состояние репозитория на ветке `arena/ee1f408a-rus-slam`; карта описывает прежде всего то, что связано исходниками, а не подтверждает готовность физического робота. **Связанные документы:** README, архитектура ПО, протокол, электрика, технический паспорт, экраны и доступ.

> ## ⚠ Ревизия карты: прошивка заменена на реальную
>
> Карта была построена против **удалённой** прошивки `firmware/module_controller` (бинарные кадры `0xAA/0xBB`, собственная 6-тактная коммутация MOSFET) и `ros2_ws/ + gui/`. Этих исходников больше нет: прошивка робота — `firmware/motor_wheel` (**fw26**, текстовый протокол по USB, драйвер тяги ZS-Z11H) и `firmware/sensor_module` (пятая плата: свет, замок отсека, карты доступа), рабочий софт — в danilka-revin/Zapret. Что перенесено оттуда и какие сведения о роботе уточнены — `ZAPRET_SYNC.md`.
>
> Поэтому:
>
> - строки карты, где упоминаются `module_controller.ino`, «6-секторная коммутация», «300 мс watchdog», «PWM ±1000», кадры 10/16 Б и MOSFET-инвертор, относятся к **истории** и помечены ниже;
> - актуальные причинные связи «прошивка → сервер» описаны в `SERIAL_PROTOCOL.md` и `firmware/README.md`;
> - раздел 6 (расхождения) сохранён как список **уроков**: большинство P1-пунктов про ROS 2 и GUI остаются в силе, пока этот стек не переведён на протокол fw26 (см. `SOFTWARE_ARCHITECTURE.md`, §10).

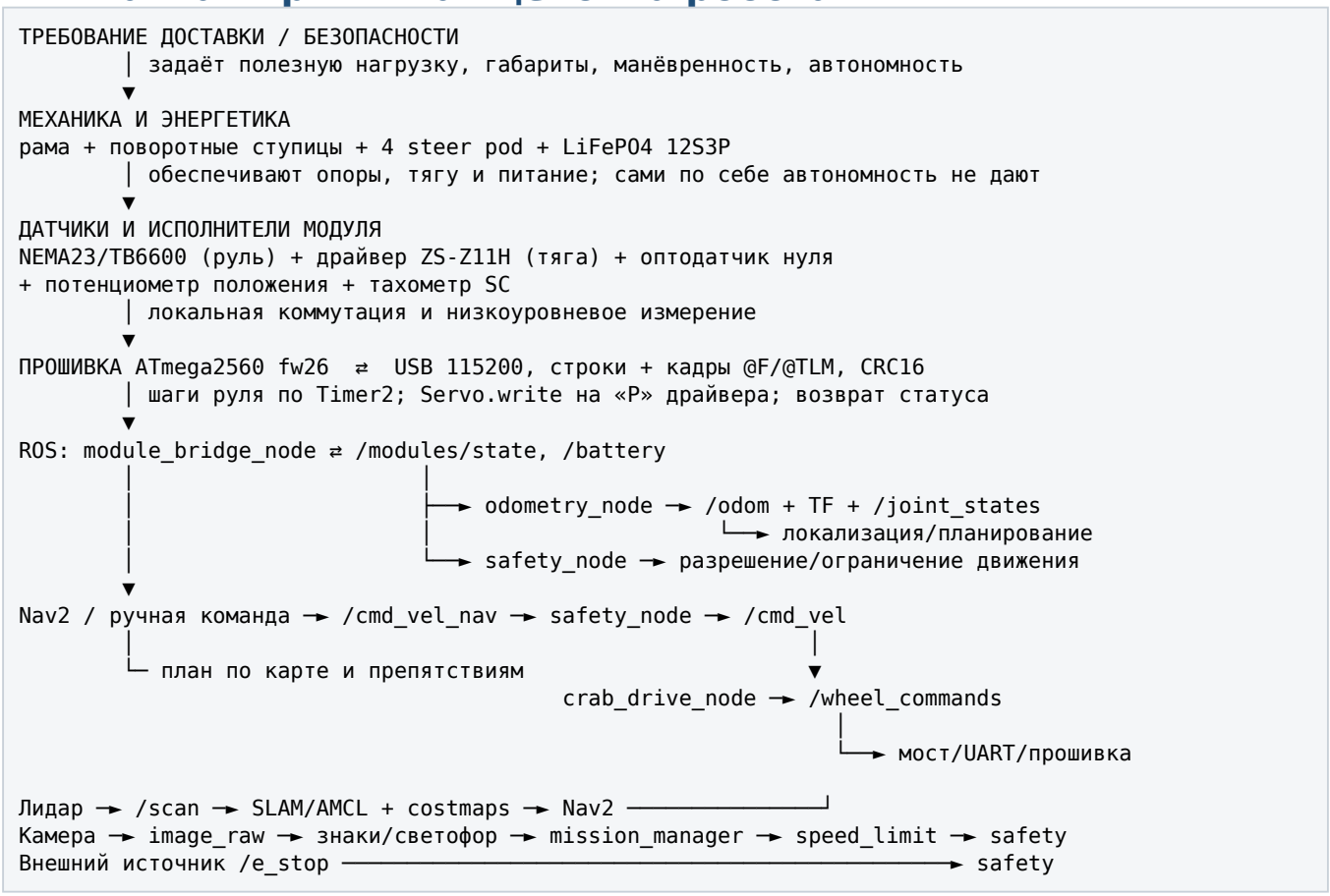
1. Короткий вывод

В репозитории есть **две разные причинно-следственные системы**, которые важно не смешивать:

1. **Бортовой контур ROS 2** задаёт программный маршрут от Nav2 до приводов и обратную связь через энкодеры, одометрию и лидар. Он описан и частично реализован в `ros2_ws/` и `firmware/`.
2. **GUI и экран борта**: инженерный пульт — в основном самостоятельный браузерный симулятор; основной экран имеет веб-клиент и отдельный нативный Tkinter-клиент без HTML. Оба могут читать API PIN-замка и источники `sim`/`serial`/`ros`, но команды движения или ручные кадры GUI в ROS/UART по текущему коду не отправляются.

Следовательно, архитектурная цепочка «навигация → моторы» существует внутри ROS 2, а цепочка «кнопка в веб-пульте → физические моторы» **не замкнута**. Дополнительный критический нюанс: `robot.yaml` включает `sim_on_fail: true`, поэтому отсутствующие UART-порты могут превратиться в четыре виртуальных модуля и пройти проверку safety как здоровые. Некоторые заявленные функции — физический E-STOP, проверенный протокол конкретного лидара, ПИД-регулирование, нагрузка 250 кг — нельзя считать доказанными только по документации и симуляции.

2. Главная причинная цепочка робота



Ссылки на первичные точки реализации: файлы прошивки — `firmware/motor_wheel/motor_wheel.ino` и `firmware/README.md`; формат кадров — `docs/SERIAL_PROTOCOL.md`; ROS-граф — `docs/SOFTWARE_ARCHITECTURE.md` и `ros2_ws/src/rus_slam_bringup/launch/`; рабочий софт — репозиторий Zapret (`slam_gui/backend.py`, `robot_control/`).

2.1. Командный поток: причина → действие

Шаг	Причина / вход	Преобразование	Непосредственный эффект
1	Nav2 публикует Twist на `/cmd_vel_nav`	`safety_node` проверяет свежесть телеметрии, аварийный стоп, неисправности, напряжение и лимит миссии	публикует ограниченную команду на `/cmd_vel`, либо нулевую
2	`/cmd_vel`	`crab_drive_node` применяет обратную кинематику по координатам четырёх модулей; команды старше `cmd_timeout_sec` превращаются в останов	`/wheel_commands`: для каждого модуля угол, скорость и enable
3	`/wheel_commands`	`module_bridge_node` переводит угол в градусы со знаком, скорость — в `v 0…150`; уничтожение устаревшего задания	отдельная **текстовая** строка на USB-порт своего колеса
4	Текстовая команда	плата проверяет адресацию и состояние (ноль, блокировки), угол превращается в шаги руля; `v` идёт в `Servo.write` драйвера ZS-Z11H	импульсы PUL/DIR/ENA на TB6600 и скважность на входе «P» драйвера тяги
5	Питание и механика исправны	шаговик поворачивает модуль, BLDC создаёт тягу на колесе	фактическое перемещение робота; оно зависит также от сцепления, нагрузки, тока и калибровки

Важное следствие: **выход обратной кинематики — ещё не движение**. Для движения должны последовательно работать safety-гейт, публикация ROS, мост, CRC/совпадение ID, прошивка, питание и силовые каскады.

2.2. Обратная связь: измерение → оценка → новое решение

Измерение	Путь данных	На что влияет
Угол руля, цель, тяга, SC, ошибки калибровки	потенциометр A0 и счётчик шагов → @F/@TLM → парсер сервера	состояние модулей, признаки потери связи, запреты движения
Фронты SC и фактический угол	телеметрия → `/modules/state` → `SwerveOdometry.estimate_twist()` (в ROS 2) или одометрия Zapret	скорость корпуса, оценка пути; SC не имеет знака
LaserScan	UART-лидар → `lds01_driver_node` → `/scan`	SLAM-карта или costmap препятствий; далее изменяется план Nav2
Цвета/форма в кадре	камера → OpenCV-детектор → `/perception/signs` и `/perception/traffic_light`	`mission_manager_node` публикует коэффициент ограничения скорости
Состояние модулей, АКБ, `/e_stop`	подписки safety-гейта	разрешение движения, ползущий режим либо нулевая команда

Одометрия не измеряет абсолютную позу: она интегрирует движение колёс, поэтому систематическая ошибка радиуса, тиков/оборот, проскальзывание или пропуски измерений накапливаются в `/odom`. SLAM/AMCL затем используют этот одометр вместе с лидаром и TF.

3. Навигация, восприятие и безопасность: причинные связи

Навигация

- `slam_toolbox.yaml` получает `/scan` и использует `odom`, `base_link`, чтобы оценивать карту `map`.
- В режиме готовой карты `navigation.launch.py` запускает map server, AMCL, controller, planner, behavior server, BT navigator и lifecycle manager.
- Успешный план Nav2 порождает скорость, а не прямые команды двигателям. Скорость должна идти через safety-гейт и затем 4WIS-кинематику.
- `mission.yaml` задаёт список точек в координатах карты. Точки — конфигурация-заготовка, а не результат картирования конкретного цеха.
- `full.launch.py` всегда включает менеджер миссии. При `slam:=true` запускается SLAM, но navigation launch включается только при `slam:=false`; значит маршрутный action server во время одного лишь картирования не предоставлен.

Восприятие

```
/camera/image_raw
   ├── HSV + форма контуров → stop / pedestrian_crossing / speed_bump
   └── цветовые пятна в верхней части кадра → red / yellow / green / unknown
                          │
                          ▼
                 mission_manager_node
                          │
          stop=0 на hold; неровность=0.3; красный=0
                          ▼
        /safety/speed_limit → safety_node → /cmd_vel
```

`camera_node` публикует `bgr8`; `vision_common.py` принимает `bgr8` и `rgb8`. Детектор знаков сегментирует цвет и аппроксимирует контур, а не запускает описанную в документах нейросеть. Детектор светофора выбирает цветовую область по площади. Поэтому качество реального распознавания зависит от экспозиции, освещения, калибровки и формы объектов;

синтетическая цветовая зона не является доказательством точности на полигоне.

Safety-гейт и останов

1. Внешний `/e_stop=true`, отсутствие свежего массива модулей, fault-модуль либо свежая критическая батарея → `safety_node` публикует нулевой `Twist`.
2. Низкое, но не критическое напряжение → умножение скоростей на `creep_limit`.
3. Устаревшая команда навигации → safety публикует нулевую скорость; затем watchdog кинематики, мостового ПО и МК дают следующие независимые ступени останова.
4. Протухшая телеметрия одного модуля в мосте отмечается fault; отсутствие четырёх модулей не даёт safety-гейту перейти в OK.

Это программная цепь ROS. Она не заменяет отдельную, физически проводную цепь силового аварийного отключения. В исходниках есть подписчик `/e_stop`, но в репозитории не найден узел/драйвер кнопки, который публикует этот Bool; в прошивке нет отдельного входа аварийного стопа.

4. Второй контур: веб-пульт и экран борта

4.1. Что происходит в браузере и в нативном основном экране

```
index.html
 ├─ console-core.js: модель АКБ, CRC/кадры, PIN/аудит, статистика
 ├─ vision.js: виртуальный цех, лучи, occupancy grid и рендер
 ├─ app.js: единственное локальное состояние, ввод, виртуальная физика и миссия
 └─ console.js: сервисная DOM-панель поверх того же симулятора

main.html + main.js ── HTTP API ── gui/backend.py
                                    ├─ отдача статики /api/state
                                    ├─ серверная модель PIN и аудит
                                    └─ источник sim / read-only serial / экспериментальный ros

native_display.py + native_display_core.py (Tkinter Canvas, без HTML)
 ├─ GET /api/state, /api/audit ── HTTP JSON ──► тот же backend.py
 └─ POST /api/lock/open|close ──────────────► та же серверная логика PIN
```

Нативный клиент не открывает страницу и не использует браузерный движок. Он рисует основной экран сам, показывает явную потерю связи и отключает действия с замком при недоступном API. Образ Windows `.exe` или Linux-бинарник собирает PyInstaller на целевой ОС; сервер остаётся отдельным процессом.

`app.js` создаёт движение и датчики внутри браузера. `vision.js` не читает физический лидар: он бросает лучи в `WORLD.walls`/`WORLD.racks`, обновляет искусственную карту и рисует камеры. `console-core.js` умеет корректно **сформировать** кадр, а `console.js` показывает HEX и применяет задание к модели сервиса. Это не равно отправке кадра через `/dev/ttyUSB*`.

`backend.py` поддерживает реальные HTTP-операции замка (при настройке внешней команды) и чтение UART-телеметрии. В нём нет API отправки `WheelCommand`, ручного PWM или UART-команды. `gui/README.md` и `docs/GUI.md` прямо называют пульт автономным симулятором; текущие источники подтверждают это.

4.2. Экран основного робота и сервисная вкладка

- `main.js` (веб-клиент) опрашивает `api/state`, `api/audit` и вызывает `api/lock/*`; при недоступности сервера показывает локальную демонстрацию.
- `native_display.py`/`native_display_core.py` (нативный Tkinter-клиент) опрашивает те же API, не грузит HTML и при потере API блокирует действия замка, чтобы не изображать неподтверждённый успех.
- `backend.py` является авторитетным хранилищем PIN на борту: соль + SHA-256, счётчик попыток, блокировка и аудит в `gui/state/lock.json`.

- `console.js` использует API бэкенда для замка, если доступен тот же HTTP-сервер. Остальная ручная ходовая команда — локальная модель и журнал HEX.
- `deploy/systemd/rus-slam-server.service` поднимает backend; для дисплея выбирают либо браузерный `rus-slam-display.service`, либо нативный `rus-slam-native-display.service`. Это независимые процессы, оба клиента подключаются к API сервера.

4.3. ROS-источник GUI — отдельный нестыкованный интерфейс

`RosSource` в `gui/backend.py` подписывается на `/screen/state`, `/battery/voltage`, `/battery/current`, `/screen/motors`. ROS-контур публикует `/modules/state` и `BatteryInfo` на `/battery`; в репозитории нет публикаций `/screen/*` и нет адаптера `ModuleStateArray` → формат экрана. При установленном `rclpy` GUI считает ROS-source запущенным, но остаётся без реальных экранных данных. Отсутствие `rclpy` переключает источник в симуляцию, но наличие библиотеки без требуемых топиков — нет.

5. От требований к файлам: карта артефактов

Артефакт	Причинная роль — что заставляет / что подтверждает
`README.md`	Точка входа: перечисляет подсистемы и команды сборки/запуска; сводит вместе спецификацию, аппаратную часть, ROS и GUI. Заявление из README само по себе не доказывает стендовое испытание.
`.gitignore`	Исключает ROS build/install/log, Python-кэши, виртуальные окружения, телеметрию/карты, `gui/state` и PyInstaller build/dist; отделяет исходную конфигурацию от генерируемого и персонального состояния.
`docs/TECHNICAL_SPECIFICATION.md`	Задаёт механические/энергетические параметры и расчётные обоснования; эти значения должны совпадать с `robot.yaml`, Xacro, протоколом и реальным измерением.
`docs/EXPLANATORY_NOTE_AVTOVAZ.md`	Обосновывает соответствие конкурсному сценарию, методики испытаний и экономический эффект; определяет функциональные цели, но не реализует алгоритмы.
`docs/ELECTRICAL_AND_PINOUTS.md`	Связывает источник 12S3P с DC-DC, Arduino-пинами, TB6600, BLDC и разъёмами; служит монтажной причиной для соответствия проводки коду.
`docs/SERIAL_PROTOCOL.md`	Норматив межслойной границы: формат команд/телеметрии, масштаб единиц, CRC, таймауты и udev. Любое несовпадение здесь ломает обмен независимо от ROS-логики.
`docs/SOFTWARE_ARCHITECTURE.md`	Заявляет ROS-граф, роли узлов, TF, topic-flow и процедуры запуска; это карта замысла, которую нужно сопоставлять с launch и кодом.
`docs/GUI.md`, `gui/README.md`	Описывают область GUI, его симуляционный характер, страницы, ввод и тестирование; помогают не принимать визуальную модель за бортовое управление.
`docs/MAIN_SCREEN.md`, `docs/RUN_AND_ACCESS.md`	Объясняют основной экран, API, PIN, роли пользователей и systemd-доступ; причинно связаны с `main.js`, `backend.py`, `deploy/`.
`docs/SERVICE_CONSOLE.md`	ТЗ и описание сервисного UI: PIN, ручные действия, АКБ, статистика, интерлоки; часть требований пока реализована только на стороне GUI-симулятора.
`docs/pdf/*.pdf`	Девять существующих PDF являются производными копиями девяти исходных MD; редактируемый источник — Markdown, сборщик — `tools/pdf/md2pdf.py`. После добавления этой карты собирается десятый PDF.

Артефакт	Причинная роль — что заставляет / что подтверждает
`docs/assets/images/system_architecture.svg`	Иллюстрирует верхнеуровневую архитектуру; содержит заявления про PID, которые расходятся с текущей прошивкой.
`docs/assets/images/kinematics_diagram.svg`	Иллюстрирует геометрию swerve и направления векторов; соответствует уравнениям `kinematics.py` по смыслу, но не задаёт параметры рабочего узла.
`docs/assets/images/module_circuit.svg`	Визуализирует электрические блоки; для реального монтажа первичны pinout, BOM, netlist и электрическая проверка.
`docs/assets/images/pod_assembly.svg`	Иллюстрирует вертикальный стек шкив → ступица → вилка → мотор-колесо; не заменяет рабочий чертёж.
`docs/assets/images/arduino_mega_pro.jpg`, `arduino_mega_pro.png`	Иллюстративные изображения контроллера, не CAD-модель и не проверка исполнения платы.
`docs/assets/images/belt_reduction_drive.png`, `nema23_motor.jpg`, `tb6600_driver.jpg`, `subaru_hub_bearing.jpg`, `scooter_hub_motor.jpg`	Иллюстрации механических и электрических компонентов, связь с расчётами паспорта/электрики.
`docs/assets/images/bldc_mosfet_driver.jpg`, `lifepo4_32700.png`, `swerve_drive_chassis.png`	Иллюстрации драйвера, аккумуляторной химии и шасси; не являются доказательством завершённой платы или испытаний.
`docs/assets/images/chassis_top.jpg`, `chassis_side.jpg`, `chassis_perspective.jpg`	На них ссылается технический паспорт как на фото изготовленной рамы; файлов в текущем checkout нет, поэтому доказательная ссылка сейчас битая.
`firmware/motor_wheel/motor_wheel.ino`	**Единственный источник истины по прошивке колеса (fw26).** Причинная связь между текстовой командой по USB и шагами руля (Timer2 → TB6600), сигналом Servo на вход «P» драйвера ZS-Z11H, оптодатчиком D18, потенциометром A0 и телеметрией `@F`/`@TLM`. Физическое поведение зависит от платы и калибровки.
`firmware/reference/steering_tested/steering_tested.ino`	Эталон, проверенный на реальной механике: пины, уровни и `133.33` шага/град сверяются с ним тестами; любая правка ядра руля обязана совпасть с этим ориентиром.
`firmware/tools/avr_build.py`, `firmware/scripts/avr_upload.py`	Сборка и заливка без Arduino IDE (avr-gcc + avrdude, ядро и Servo с GitHub): причина, по которой прошивку вообще можно воспроизвести там, где `downloads.arduino.cc` недоступен.
`firmware/scripts/firmware_probe.py`, `firmware/scripts/flash_firmware.sh`	Читают `fw=` с плат и заливают прошивку по `RC_WHEEL_*_PORT`; **не являются** автопоиском ролей — требуют подтверждения человеком.
`firmware/firmware_sim/`	Хост-симуляция четырёх плат (TB6600, оптодатчик, потенциометр, EEPROM) на настоящем скетче: 61 сценарий проверяет логику до выезда; отдельная симуляция модуля датчиков — 20 сценариев. Не доказывает уровни на реальных выводах.

Артефакт	Причинная роль — что заставляет / что подтверждает
pcb/README.md, pcb/STATUS.md	Инженерный архив v1.0: перечисляет артефакты и **явно фиксирует расхождение пинов с прошивкой колеса (fw26)** (TB6600 D6/D7/D8, Servo D3, опто D18, потенциометр A0).
pcb/SCHEMATIC_CIRCUIT_DESIGN.md	Описывает силовой каскад и расчёты; требует сверки с фактической схемой/DRC и компонентами.
pcb/BOM.csv	Связывает позиционные обозначения с компонентами и артикулами; покупка BOM не гарантирует правильную распайку или электрическую безопасность.
pcb/CONNECTOR_PINOUTS.csv	Связывает контакты, цвета, напряжения и сигналы с кодовыми пинами; должен совпасть с прошивкой.
pcb/kicad_netlist.net	Список цепей и позиционных обозначений; не содержит готовой трассировки платы и полного производственного проекта.
pcb/easyeda_schematic_guide.md	Порядок переноса BOM/netlist в САПР и изготовления; производственная операция ещё требует проектирования/проверки DRC.
ros2_ws/src/rus_slam_interfaces/msg/*.msg, srv/*.srv	Схемы контрактов между узлами: команды колёс, состояние модулей/АКБ, восприятие, хоминг и enable. Изменение типа требует согласованных изменений издателя и подписчика.
rus_slam_interfaces/CMakeLists.txt, package.xml	Причина генерации и сборки ROS-интерфейсов; CMake создаёт типы, package manifest задаёт зависимости.
rus_slam_base/protocol.py	CRC16, сборка команд и потоковый разбор телеметрии: проверяемая программная граница Python ↔ прошивка.
rus_slam_base/module_link.py	Реальная функция открытия UART и имитационная модель модуля; делает возможными отдельные режимы железа и sim.
rus_slam_base/module_bridge_node.py	Перевод ROS-команд в UART и кадров обратно в /modules/state//battery; также создаёт программную подмену при sim_on_fail.
rus_slam_base/kinematics.py, crab_drive_node.py	Чистый алгоритм обратной кинематики и ROS-обёртка, превращающие /cmd_vel в /wheel_commands.
rus_slam_base/odometry_core.py, odometry_node.py	Обратная оценка twist и позы по измерениям колёс; публикует /odom, TF и суставы.
rus_slam_base/test/test_protocol.py, test_kinematics.py, test_odometry.py	Проверяют эталон CRC, преобразование команд и математические сценарии; не запускают ROS executor, UART или двигатель.
rus_slam_lidar/lds_protocol.py, lds01_driver_node.py	Разбор UART-лидара и упаковка в LaserScan; здесь сенсорная спецификация становится причиной данных SLAM/Nav2.

Артефакт	Причинная роль — что заставляет / что подтверждает
`rus_slam_lidar/test/test_lds_protocol.py`	Проверяет parser на кадре, построенном самим тестом, включая тот же XOR checksum; не является испытанием реального LDS-01.
`rus_slam_perception/vision_common.py`	Преобразование ROS Image в BGR, импорт OpenCV и аппроксимация формы; общий нижний уровень двух детекторов.
`rus_slam_perception/camera_node.py`	Источник image_raw/camera_info; камера отсутствует — изображения не публикуются.
`rus_slam_perception/sign_detector_node.py`, `traffic_light_node.py`	Причинные преобразователи кадра в классы знаков и цвет сигнала для менеджера миссии.
`rus_slam_safety/safety_node.py`	Единственный ROS-гейт между запросом скорости и `/cmd_vel`; переводит состояния модулей/АКБ/E-stop в останов или ограничение.
`rus_slam_safety/mission_manager_node.py`	FSM доставки: readiness → homing → waypoints; реакция на safety и perception через отмену action и speed_limit.
`rus_slam_navigation/config/nav2_params.yaml`, `slam_toolbox.yaml`	Параметры AMCL, планировщика, costmap и SLAM; задают поведение, но корректность зависит от действительных TF, сенсора и карты.
`rus_slam_navigation/launch/navigation.launch.py`, `slam.launch.py`	Запуск Nav2/AMCL или SLAM; причинно определяют, какие action server и TF-коррекции вообще существуют в выбранном режиме.
`rus_slam_navigation/maps/README.md`	Инструкция получения и сохранения карты; в репозитории готового `*.yaml`/`*.pgm` цеха нет.
`rus_slam_description/urdf/rus_slam.urdf.xacro`, `description.launch.py`	Геометрия звеньев и TF для robot_state_publisher; неверные размеры/смещения искажают costmap, визуализацию и локализацию.
`rus_slam_bringup/config/robot.yaml`, `lidar.yaml`, `perception.yaml`, `safety.yaml`, `mission.yaml`	Рабочие числовые настройки транспорта, сенсоров, порогов и маршрута; это конфигурационная причина фактического поведения ROS-узлов.
`rus_slam_bringup/launch/robot.launch.py`, `full.launch.py`	Составляют ROS-граф и ветвят запуск на симуляцию, картирование или локализацию по готовой карте.
`rus_slam_bringup/udev/99-rus-slam.rules`	Именует UART-устройства по USB-пути; если шаблонные devpath не совпадут с бортом, нужный порт не откроется.
`package.xml`, `setup.py`, `setup.cfg`, `resource/*`, `__init__.py` пакетов ROS	Метаданные, entry points, установка launch/config и обнаружение пакетов colcon/ament; сами алгоритмы не реализуют.
`gui/console-core.js`	Независимое от DOM ядро АКБ, кадра, PIN и статистики; вычисляет значения для веб-пульта и unit tests.

Артефакт	Причинная роль — что заставляет / что подтверждает
`gui/console.js`	DOM-панель «Сервис», запросы к API замка и локальная имитация ручных операций/статистики.
`gui/app.js`, `gui/vision.js`	Цикл браузерной симуляции и виртуальный мир; производят локальные позу, лидар, карту, камеру, миссию и отрисовку.
`gui/index.html`, `gui/styles.css`	Структура и оформление семи вкладок инженерного пульта; CSS меняет представление, не подключение к приводам.
`gui/main.html`, `main.css`, `main.js`	Веб-клиент основного экрана; отображает источник данных и вызывает API PIN-замка.
`gui/native_display.py`, `native_display_core.py`	Нативный экран Tkinter/Canvas без HTML и браузера; читает JSON API, PIN-команды идут в тот же backend; PyInstaller-сборка зависит от целевой ОС.
`gui/backend.py`	HTTP/API, серверная PIN-база, sim-source, UART read-only-source и экспериментальный ROS-source; общий backend для веб- и нативного экрана.
`gui/serve.py`	Точка совместимости, передающая аргументы в `backend.main`.
`gui/tests/console.test.js`	Чистые проверки ядра `console-core.js` без DOM.
`gui/tests/jsdom.console.test.js`, `main.screen.test.js`, `ui.audit.test.js`	Проверки DOM сервиса/основного экрана и статического UI-аудита.
`gui/tests/backend.test.py`	Unit/API проверки backend, источников, CRC, PIN и аудита.
`gui/tests/native_display.test.py`	Проверяет JSON-клиент нативного экрана против живого локального API, обработку PIN и состояния.
`gui/tests/console.api.test.js`, `main.api.test.js`	Сквозные браузерные проверки между страницей и HTTP backend; не поднимают ROS или реальные UART-модули.
`deploy/systemd/rus-slam-server.service`, оба display service, `deploy/systemd/display.sh`, `deploy/README.md`	Делают backend доступным и запускают один выбранный экран; не запускают ROS-робота и не управляют ходовыми приводами.
`tools/pdf/md2pdf.py`, `tools/pdf/requirements.txt`	Генерируют документальные копии Markdown и проверяют наличие/время PDF; не тестируют код системы.

5.1. Полный состав файлов checkout по слоям

Для исключения неоднозначности ниже перечислены все tracked-файлы по их месту в репозитории; имена уже подробно разобранных функциональных файлов не повторяют описание из таблицы выше.

- **Корень:** `.gitignore`, `README.md`.
- **Развёртывание:** `deploy/README.md`, `deploy/systemd/{display.sh,rus-slam-display.service,rus-slam-native-display.service,rus-slam-server.service}`.

- **Документы-источники:** docs/ELECTRICAL_AND_PINOUTS.md, EXPLANATORY_NOTE_AVTOVAZ.md, GUI.md, MAIN_SCREEN.md, RUN_AND_ACCESS.md, SERIAL_PROTOCOL.md, SERVICE_CONSOLE.md, SOFTWARE_ARCHITECTURE.md, TECHNICAL_SPECIFICATION.md и эта карта CAUSAL_MAP.md.
- **Производные PDF:** docs/pdf/ELECTRICAL_AND_PINOUTS.pdf, docs/pdf/EXPLANATORY_NOTE_AVTOVAZ.pdf, docs/pdf/GUI.pdf, docs/pdf/MAIN_SCREEN.pdf, docs/pdf/RUN_AND_ACCESS.pdf, docs/pdf/SERIAL_PROTOCOL.pdf, docs/pdf/SERVICE_CONSOLE.pdf, docs/pdf/SOFTWARE_ARCHITECTURE.pdf, docs/pdf/TECHNICAL_SPECIFICATION.pdf, docs/pdf/CAUSAL_MAP.pdf.
- **Иллюстрации:** docs/assets/images/arduino_mega_pro.jpg, arduino_mega_pro.png, belt_reduction_drive.png, bldc_mosfet_driver.jpg, kinematics_diagram.svg, lifepo4_32700.png, module_circuit.svg, nema23_motor.jpg, pod_assembly.svg, scooter_hub_motor.jpg, subaru_hub_bearing.jpg, swerve_drive_chassis.png, system_architecture.svg, tb6600_driver.jpg. Отсутствующие chassis_top.jpg, chassis_side.jpg, chassis_perspective.jpg являются ссылками, но не файлами checkout.
- **Прошивка:** firmware/README.md, firmware/motor_wheel/motor_wheel.ino, firmware/reference/README.md, firmware/reference/steering_tested/steering_tested.ino, firmware/tools/avr_build.py; тестовая обвязка firmware/firmware_sim/{arduino_sim.h,board_prelude.h,sim.cpp,stubs/*}; утилиты firmware/scripts/{avr_upload.py,firmware_probe.py,flash_firmware.sh,install_arduino_cli.sh,sync_firmware_desktop.sh}.
- **GUI:** gui/README.md, app.js, backend.py, tools/build-native-linux.sh, tools/build-native-windows.bat, console-core.js, console.js, index.html, main.css, main.html, main.js, native_display.py, native_display_core.py, requirements-native-display.txt, serve.py, styles.css, vision.js; тесты gui/tests/backend.test.py, console.api.test.js, console.test.js, jsdom.console.test.js, main.api.test.js, main.screen.test.js, native_display.test.py, ui.audit.test.js.
- **PCB:** pcb/README.md, BOM.csv, CONNECTOR_PINOUTS.csv, SCHEMATIC_CIRCUIT_DESIGN.md, easyeda_schematic_guide.md, kicad_netlist.net.
- **Инструменты документации:** tools/pdf/md2pdf.py, tools/pdf/requirements.txt.
- **ROS 2 —** rus_slam_base**:** package.xml, resource/rus_slam_base, setup.cfg, setup.py; rus_slam_base/__init__.py, crab_drive_node.py, kinematics.py, module_bridge_node.py, module_link.py, odometry_core.py, odometry_node.py, protocol.py; тесты test/test_kinematics.py, test/test_odometry.py, test/test_protocol.py.
- **ROS 2 —** rus_slam_bringup**:** package.xml, resource/rus_slam_bringup, setup.cfg, setup.py, rus_slam_bringup/__init__.py; config/lidar.yaml, mission.yaml, perception.yaml, robot.yaml, safety.yaml; launch/full.launch.py, launch/robot.launch.py; udev/99-rus-slam.rules.
- **ROS 2 —** rus_slam_description**:** package.xml, resource/rus_slam_description, setup.cfg, setup.py, rus_slam_description/__init__.py, launch/description.launch.py, urdf/rus_slam.urdf.xacro.
- **ROS 2 —** rus_slam_interfaces**:** CMakeLists.txt, package.xml; сообщения msg/BatteryInfo.msg, ModuleState.msg, ModuleStateArray.msg, SignDetection.msg, SignDetectionArray.msg, TrafficLightState.msg, WheelCommand.msg, WheelCommandArray.msg; сервисы srv/HomeModules.srv, SetModulesEnabled.srv.
- **ROS 2 —** rus_slam_lidar**:** package.xml, resource/rus_slam_lidar, setup.cfg, setup.py, rus_slam_lidar/__init__.py, lds01_driver_node.py, lds_protocol.py, test/test_lds_protocol.py.
- **ROS 2 —** rus_slam_navigation**:** package.xml, resource/rus_slam_navigation, setup.cfg, setup.py, rus_slam_navigation/__init__.py; config/nav2_params.yaml,

`config/slam_toolbox.yaml`; `launch/navigation.launch.py`, `launch/slam.launch.py`; `maps/README.md`.

- **ROS 2 —** `rus_slam_perception`**:** `package.xml`, `resource/rus_slam_perception`, `setup.cfg`, `setup.py`, `rus_slam_perception/__init__.py`, `camera_node.py`, `sign_detector_node.py`, `traffic_light_node.py`, `vision_common.py`.
- **ROS 2 —** `rus_slam_safety`**:** `package.xml`, `resource/rus_slam_safety`, `setup.cfg`, `setup.py`, `rus_slam_safety/__init__.py`, `mission_manager_node.py`, `safety_node.py`.

`package.xml`, `setup.py`, `setup.cfg`, `resource/*` и `__init__.py` — не скрытые управляющие алгоритмы: они обеспечивают обнаружение, установку, entry points и зависимости пакетов. PDF, фотографии и схемы иллюстрируют/дублируют смысл, но causal source остаётся соответствующим `.md`, `.py`, `.ino`, `.msg`/`.srv`, YAML или сетевым файлом.

6. Расхождения, которые разрывают причинную цепочку

> **Исторические строки.** Пункты, ссылающиеся на `module_controller.ino` (6-секторная коммутация, `CURRENT_FULL_MA`, таймеры D8–D13, homing без различения таймаута), относились к **удалённой** прошивке и больше не воспроизводятся. Остальные пункты — про `ros2_ws/` и `gui/` — остаются в силе, пока этот стек не переведён на протокол fw26 (см. SOFTWARE_ARCHITECTURE.md, §10). Актуальные риски прошивки: калибровка нуля после каждого перезапуска платы, необходимость `q` для доводки, `zfault` при повреждённой EEPROM, отсутствие сертифицированного E-STOP и дрейф числа переходов SC на оборот.

Приоритеты: **P1** — устранить или подтвердить до подключения к движущемуся роботу; **P2** — устранить до заявлений о готовности/приёмке; **P3** — документация/удобство. Это анализ дерева файлов, не заключение о неизвестном внешнем оборудовании.

Приоритет	Наблюдаемая причина	Следствие	Где видно
P1	`robot.yaml` задаёт `sim_on_fail: true`; когда UART-порт недоступен, `module_bridge_node` создаёт `SimulatedModule`, публикует все четыре штатных состояния, а safety не различает источник данных	С потерянным/неподключённым контроллером ROS-гейт может видеть четыре «здоровых» модуля и перейти в OK; команда при этом не дойдёт до физического мотора. Для боевого запуска нужно отключить fallback и проверять источник/идентичность каждого модуля	`robot.yaml:8–15`; `module_bridge_node.py:75–100,171–176`; `module_link.py:210–266`; `safety_node.py:98–108,123–140`
P1	GUI формирует кадр и пишет HEX в журнал, но путь отправки отсутствует; `backend.py` serial читает телеметрию, а API содержит только экран, PIN и audit	Ручная команда из «Сервиса» изменяет демонстрационную модель, а не доказанно реальный мотор	`gui/console.js` около `sendModule()`; `gui/backend.py` routes `do_GET`/`do_POST` и `SerialSource`; `docs/GUI.md` §9 говорит, что JS-моста ROS нет
P1	ROS backend подписан на `/screen/*`, но ROS публикует `/modules/state` и `/battery`; адаптера в исходниках нет	`--source ros` при установленном `rclpy` показывает пустые/подставные экранные данные, не реальную телеметрию	`gui/backend.py:368–371`; издатели `module_bridge_node.py:103–105`; поиск по репозиторию не находит publisher `/screen/*`
P1	Safety подписывается на `/e_stop`, но в launch/source нет publisher кнопки; прошивка не имеет выделенного E-stop pin	E-stop в коде действует лишь когда внешний Bool действительно подан; граф ROS не доказывает аппаратное отключение силовой цепи	`safety_node.py:79–85`; `module_controller.ino` список пинов; `robot.launch.py`

Приоритет	Наблюдаемая причина	Следствие	Где видно
P1	`/home_modules` отвечает `success` после постановки временной задачи на 1 секунду; прошивка выполняет blocking homing; затем безусловно ставит `is_homed=true`, в том числе после таймаута	Менеджер миссии может перейти в READY по факту приёма запроса, а не подтверждённого нулевого положения; HOME может повторяться при потоке кадров	`module_bridge_node.py:135–143,157–169`; `mission_manager_node.py:146–155`; `module_controller.ino:203–219,336–358`
P1	Lidar parser захардкожен на XOR; документация называет формат «совместимым с семейством», а реальный паспорт LDS-01 отсутствует; checksum-алгоритм параметром не задаётся	Реальный сенсор с иной контрольной суммой будет отвергать каждый пакет и не даст SLAM сканов	`lds_protocol.py:5–16,31–74`; `lidar.yaml`; `docs/SERIAL_PROTOCOL.md §4` прямо оставляет поля до паспорта
P1	`navigation.launch.py` переназначает `/cmd_vel` только у `controller_server`; `behavior_server` запускается без remapping	Recovery-поведение может публиковать напрямую в `/cmd_vel`, обходя safety-гейт. Требует проверки фактического topic/type в Humble и Jazzy; до проверки считать маршрутизацию неподтверждённой	`navigation.launch.py:48–72`; схема намерения — `docs/SOFTWARE_ARCHITECTURE.md`
P1	AMCL настроен на `DifferentialMotionModel`, тогда как база допускает значительную скорость vy и крабовый ход	При боковом перемещении модель движения AMCL может систематически не соответствовать одометрии и смещать локализацию	`nav2_params.yaml:36`; кинематика `kinematics.py`; режимы в `README.md`
P2	Прошивка напрямую отображает PWM на 6-секторную коммутацию; PID-регулятора скорости/угла в коде нет, encoder используется для дельты телеметрии	Замкнутый контроль скорости и позиционирование, описанные в паспорте/схемной SVG, не следует из текущей реализации	`module_controller.ino:108–159,182–200,328–344`; `TECHNICAL_SPECIFICATION.md §5`; `system_architecture.svg`
P2	В `run_homing_sequence()` результат концевика не различается от истечения 10 с; обе ветки присваивают `is_homed=true`	Неисправный/неподключённый switch может сообщить ложный успех; автомат миссии не отличает этот случай	`module_controller.ino:203–219`
P2	`lds01_driver_node` публикует LaserScan на каждый короткий UART-кадр; параметры `angle_min_deg`/`angle_max_deg` объявлены, но не участвуют в публикации; parser не фильтрует по настроенным границам	Формат, круговой охват, порядок и качество `/scan` на реальном устройстве надо проверить rosbag/RViz и реальным Nav2; синтетический тест этого не проверяет	`lds01_driver_node.py:26–35,85–117`; `lds_protocol.py`
P2	Топик `/battery` в ROS оценивает SOC линейно по 33–42 В; GUI применяет кусочно-линейную OCV-кривую; GUI ROS-source к тому же ждёт другой набор топиков	Один и тот же физический пак может давать разные проценты/уровень на ROS и экранах	`module_bridge_node.py:54–70`; `gui/backend.py:84–93,181–214`; `console-core.js:83–113`

Приоритет	Наблюдаемая причина	Следствие	Где видно
P2	`CURRENT_FULL_MA=25000` соответствует 0–25 А на полном диапазоне АЦП, в то время как расчёт схемы INA180A1 + 5 мОм + усиление 20 даёт 0,1 В/А и 2 В при 20 А	Коэффициент тока может быть вдвое неверен (в зависимости от фактического аналогового тракта); ошибочны батарейная сводка и пороговая оценка	`module_controller.ino:40–42,239–243`; `pcb/SCHEMATIC_CIRCUIT_DESIGN.md` §2.3
P2	Пины D8–D13 используют разные аппаратные таймеры; код меняет только TCCR1B и TCCR2B, а комментарий обещает 31.25 кГц всем фазам	Не все назначенные PWM-выходы автоматически получают заявленную частоту; до включения MOSFET нужно измерить осциллографом каждую линию, dead-time и отсутствие сквозного тока	`module_controller.ino:61–68,301–304`; pinout `docs/ELECTRICAL_AND_PINOUTS.md` §2
P2	Телеметрия модулей повторно публикуется циклом моста, а wheel speed считается из `enc_delta` за `cycle_dt`; odometry интегрирует каждое сообщение массива	Повтор/пропуск UART-кадра способен исказить скорость/путь; требуется проверить timestamp каждого sample и частоту реального потока	`module_bridge_node.py:156–220`; `odometry_node.py:472–492`
P2	При усечении аудита backend оставляет последние 200 записей, но проверка всегда ожидает `prev='genesis'`	После записи более 200 событий корректная обрезанная цепочка будет показана как повреждённая	`gui/backend.py:471–493`; тот же принцип в PinVault `gui/console-core.js`
P2	Конфигурация миссии содержит фиксированные точки, а карта реального цеха отсутствует; паспорт прямо описывает испытания как запланированные	Успех автономного маршрута на целевой площадке не следует из наличия Nav2/FSM и тестов кинематики	`mission.yaml`; `maps/README.md`; `TECHNICAL_SPECIFICATION.md` §10
P2	В browser-sim зона знака `stop` обновляет только текстовую подсказку; движение там явно не обнуляется. В ROS-менеджере знак ограничивается на 3 секунды	Демо GUI может визуально показать проезд мимо STOP, даже если документ описывает остановку; GUI-сценарий и ROS-политика имеют разную причинную семантику	`gui/app.js` около обработки `WORLD.signs`; `mission_manager_node.py:99–116`
P2	В `SOFTWARE_ARCHITECTURE.md` написаны `respawn:=true` и публикация пустых сканов при отказе лидара, но launch-узлы не включают respawn, а lidar node без порта scan не публикует	Документ обещает восстановление/поведение при отказе, которых текущая конфигурация не задаёт	`SOFTWARE_ARCHITECTURE.md` §8; `robot.launch.py`; `lds01_driver_node.py:48–64,67–83`
P3	В `docs/TECHNICAL_SPECIFICATION.md` есть ссылки на chassis-фото, которых нет; `README.md` заявляет фото промежуточной сборки	Часть документального подтверждения механики недоступна из checkout	`docs/TECHNICAL_SPECIFICATION.md` §3.1; отсутствуют `docs/assets/images/chassis_*.jpg`
P3	Абсолютные заявления «LiFePO4 не горит / отсутствует риск возгорания» сильнее инженерного основания; химия устойчивее NMC, но батарея/проводка/BMS всё ещё требуют защиты и испытаний	Нельзя превращать сравнительную характеристику химии в обещание нулевого пожарного риска	`README.md`, `TECHNICAL_SPECIFICATION.md` §8, `ELECTRICAL_AND_PINOUTS.md`

7. Что именно проверено тестами

В этом окружении выполнены:

- **Прошивка:** хост-симуляция четырёх плат `firmware/firmware_sim` — **61/61 сценарий** (калибровка, потенциометр, удержание, тяга ZS-Z11H, watchdog, EEPROM, телеметрия с полем ра); симуляция модуля датчиков — **20/20 сценариев** (свет, лента, карты, замок, потеря связи). Обе запускает `pytest firmware/tests` — **27 passed**. Сборка avr-gcc через `firmware/tools/avr_build.py` выполняется только при наличии тулчейна.
- 25 pure-Python тестов ROS-ядра и lidar parser: `test_kinematics.py`, `test_odometry.py`, `test_protocol.py`, `test_lds_protocol.py` — **25 passed**.
- GUI: `console.test.js` — 32; backend `test.py` — 40; jsdom `jsdom.console.test.js` — 93; `main.screen.test.js` — 38; `ui.audit.test.js` — 20; `console.api.test.js` — 17; `main.api.test.js` — 20; новый `native_display.test.py` — 9. Всего по GUI/API — **269 passed**.
- Нативный экран и его API-клиент проверены на синтаксис Python, а Linux build-script — `bash -n`; `tools/pdf/md2pdf.py --check` подтверждает актуальность PDF для 12 Markdown-документов.
- Смета: `pytest tools/estimate` — **30 passed** (арифметика, фотографии и источники, совпадение итогов в документах).

Эти тесты подтверждают внутреннюю арифметику, сериализацию, поведение API замка и DOM. **Нативный экран не был визуально запущен, а .exe/PyInstaller-сборка не создавалась**: в текущем окружении нет Tkinter/графической сессии и Windows. Тесты не подтверждают сборку ROS 2/colcon, запуск Nav2, **заливку прошивки на реальную плату**, правильность пинов на конкретной механике, реальный LiDAR, физический E-STOP, реальные обороты SC, грузоподъёмность 250 кг или прохождение полигона. В окружении нет `ros2`, `colcon` и `arduino-cli`; документация технического паспорта описывает натурные тесты как планируемые.

8. Рекомендуемый порядок замыкания причинных цепочек

1. **Сначала аппаратные безопасные условия:** проверить физическую схему аварийного отключения, предохранители, BMS, драйвер затворов, dead-time, токовую калибровку и температуру; не подключать нагруженные моторы только по документации.
2. **Прошивка:** прошить **одну** плату с ролью FL (`firmware/README.md`), проверить `p` (`fw=26`) и `fw=` из `firmware/scripts/firmware_probe.py`, затем `k` и `q` на вывешенном колесе; сверить 133.33 шага/град транспортиром при `d 90 f`, направление `v 3 f/b` и уровни STOP/BRAKE осциллографом; только после этого — остальные три платы.
3. **Хоминг и телеметрия:** проверить, что `k` ставит `cal=1` и `home=opto` (а не таймаут), что `q` сохраняется после перезапуска платы, и что `sc_ppr=90` соответствует реальному числу переходов на оборот вашего мотора.
4. **ROS без тяги:** собрать workspace, проверить типы, частоты, freshness, TF, `/scan`, `/odom`, `map→odom`; запустить симуляцию только явным `sim:=true`. Для физического робота отключить `sim_on_fail`, иначе отказ UART маскируется виртуальным модулем; затем проверить Nav2 endpoints.
5. **Проверить, что все velocity publishers проходят через safety:** в частности recovery behaviors; при любой неоднозначности remap-ить их на `/cmd_vel_nav`, а `/cmd_vel` оставить только выходом safety.
6. **Замкнуть GUI только отдельным мостом:** выбрать контракт команд/состояния, аутентифицировать и ограничить ручные операции, затем подключить web UI к этому мосту; не считать HEX-предпросмотр отправкой команды.
7. **Только потом — натурные испытания:** торможение, повороты, crab, круговой разворот, потеря каждого UART, stale battery, красный свет, знак «Стоп», нагрузка, нагрев, связь и

повторные рейсы; результаты записывать отдельно от планов.

9. Итоговая причинно-следственная формула

```
Робот движется безопасно
⇔ механика и питание исправны
 ∧ физические измерения корректны и свежи
 ∧ контракт UART совпадает на ПК и МК
 ∧ safety получает реальные данные и закрывает все velocity-пути
 ∧ кинематика/одометрия/TF калиброваны
 ∧ навигация получает валидные scan/map/TF
 ∧ исполнительный каскад подтверждён испытанием.
```

Наличие каждого файла в дереве доказывает наличие артефакта, но не автоматически истинность всего конъюнктивного условия. Наиболее важные причины возможного разрыва сейчас — незамкнутый GUI→аппаратный command path, неподключённый источник `/e_stop`, неподтверждённая связь протокола лидара с экземпляром LDS-01 и недостоверное подтверждение хоминга.